Universidad de Antioquia

Facultad de Ciencias Exactas y Naturales

Examen 1: Fundamentos en ciencia de datos (FCD) 20 Feb 2025

- Nombre :

- E-mail :

- Profesor laboratorio :

Instrucciones del Examen

Este examen debe realizarse de manera estrictamente **individual**. Lea atentamente las siguientes reglas antes de comenzar:

- **Tiempo límite:** La duración del examen es de 90 minutos (1 hora 30 minutos).
- **Formato de respuestas:**
  - Preguntas con **círculos** (◯) permiten seleccionar **una única respuesta**.
  - Preguntas con **cuadros** [☐] permiten seleccionar **múltiples respuestas**.
- **Uso de materiales externos:** Está completamente prohibido el uso de libros, apuntes, dispositivos electrónicos como calculadoras, teléfonos o computadoras, consultas a otras personas o cualquier forma de colaboración.

Formato del examen:

- Responda directamente en la hoja del examen en los espacios designados. Asegúrese de que sus respuestas sean legibles y definitivas; las respuestas ambiguas o tachadas pueden no ser evaluadas.

Normas de conducta:

- Cualquier intento de fraude o comunicación con otros estudiantes resultará en la anulación inmediata del examen.
- En caso de dudas, consulte únicamente con el supervisor del examen.

Entrega: Al finalizar el tiempo, entregue el examen de inmediato. No se aceptarán exámenes fuera del tiempo asignado.

Lea cuidadosamente cada pregunta y conteste con claridad. ¡Éxito en su examen!

1. (23 puntos) Verdadero o Falso

1. (1.0 pt) Todo el trabajo en este examen es exclusivamente tuyo.

   ◯ Verdadero ◯ Falso

2. (2.0 pt) La probabilidad de que dos eventos A y B ocurran juntos puede, en ocasiones, ser mayor que la probabilidad de que ocurra solo A o solo B.

   ◯ Verdadero ◯ Falso

3. (2.0 pt) La Ley de los Grandes Números implica que con alta probabilidad, la distribución empírica de una gran muestra aleatoria se parecerá a la distribución de la población de la que se extrajo.

   ◯ Verdadero ◯ Falso

4. (2.0 pt) La distancia de variación total se puede aplicar a distribuciones categóricas en las que solo hay dos categorías posibles (por ejemplo, rojo o azul).

   ◯ Verdadero ◯ Falso

5. (2.0 pt) En una prueba A/B, la razón por la que se barajan las etiquetas es para garantizar que los sujetos sean asignados aleatoriamente a los grupos de tratamiento y control.

   ◯ Verdadero ◯ Falso

6. (2.0 pt) Supongamos que se propone una prueba de hipótesis y ya sabemos que la hipótesis nula es verdadera. Si 500 investigadores toman muestras independientes del mismo tamaño y usan un umbral de 1% para el valor p, se espera que aproximadamente 5 de ellos rechacen la hipótesis nula.

   ◯ Verdadero ◯ Falso

7. (2.0 pt) La altura de cada barra en un histograma siempre representa la proporción de datos dentro del intervalo correspondiente.

   ◯ Verdadero ◯ Falso

8. (2.0 pt) Si eres un sujeto en un experimento, saber si estás en el grupo de tratamiento o control puede considerarse una variable de confusión.

   ◯ Verdadero ◯ Falso

9. (2.0 pt) Una función en Python siempre debe tener una declaración de retorno (`return`).

   ◯ Verdadero ◯ Falso

   El siguiente código simula el lanzamiento de una moneda equilibrada:

```
import numpy as np
import  datascience

def simulacion_moneda_caras():
    moneda = make_array("cara", "sello")
    lanzamientos =  np.random.choice(moneda, 100)
```

```
    return np.count_nonzero(lanzamientos=="cara")

num_repeticiones = 20000
cantidad_caras = make_array()

for i in np.arange(1, num_repeticiones):
    nuevo_valor = simulacion_moneda_caras()
    cantidad_caras = np.append(cantidad_caras, nuevo_valor)

cantidad_sellos = 100 - cantidad_caras
```

Responda si las siguientes afirmaciones son **Verdaderas** o **Falsas**, basándose en el código anterior.

10. (2.0 pt) El código simula el lanzamiento de una moneda 100 veces en cada iteración y repite este proceso 20,000 veces, almacenando los resultados.

    ◯ Verdadero ◯ Falso

11. (2.0 pt) La función `simulacion_moneda_caras()` cuenta cuántas veces aparece "cara" en 100 lanzamientos y devuelve ese número.

    ◯ Verdadero ◯ Falso

12. (2.0 pt) La variable `cantidad_sellos` contiene una lista con la cantidad de veces que salió "sello" en cada simulación, calculada restando `cantidad_caras` de 100.

    ◯ Verdadero ◯ Falso

2. (24 puntos) Completa el espacio en blanco

Un grupo de amigos, Luis, Ana y Carlos, están en una tarde de juegos y deciden realizar un experimento de probabilidad. Encuentran una bolsa con exactamente tres bolas de diferentes colores:

- 1 bola roja
- 1 bola verde
- 1 bola azul

Curiosos por explorar conceptos de probabilidad, deciden jugar con las siguientes reglas:

- Mezclan bien las bolas dentro de la bolsa.
- Extraen una bola y anotan su color.
- La bola **no** se regresa a la bolsa.
- Mezclan nuevamente las bolas restantes y extraen una segunda bola.
- Registran el color de la segunda bola extraída.
- Finalmente sacan la última bola y registran su color

Antes de jugar muchas rondas, se preguntan: ¿cuál es la probabilidad de ciertos eventos? Y como buenos entusiastas de la programación, también quieren escribir un código en Python para simular el experimento.

(a) (12.0 puntos) Cálculo de probabilidades

Responde a las siguientes preguntas sobre la probabilidad de ciertos eventos en este experimento:

1. (3.0 pt) Representemos cada color de bola con una letra:

   - **R** para la bola roja
   - **V** para la bola verde
   - **A** para la bola azul

   Escribe el **espacio muestral** del experimento, es decir, todas las posibles combinaciones de bolas que pueden ser extraídas en orden.

2. (3.0 pt) ¿Cuál es la probabilidad de que una de las tres bolas extraídas sea **verde** y otra sea **roja**, sin importar el orden?

3. (3.0 pt) Si repetimos este experimento 500 veces, ¿cuántas veces esperaríamos obtener una bola verde y una roja en cualquier orden?

4. (3.0 pt) ¿Cuál es la probabilidad de que la última bola extraída sea la **azul**?

El grupo quiere simular cuántas veces la bola azul es la última en ser extraída en varias rondas de extracción **sin reemplazo**. **Luis** escribe el siguiente código incompleto:

```
import numpy as np
from datascience import make_array

def simular_bola_azul(n):
    colores = make_array("roja", "verde", "azul")
    conteo_exitos = 0

    for _ in np.arange(n):
      # Extrae las tres bolas
        muestra = np.random.choice(colores, 3, replace=False)sin reemplazo
        if __________:  # (a)
            conteo_exitos += 1

    return conteo_exitos / n
```

1. (2.0 pt) Completa el espacio **(a)** con una condición lógica que verifique si la bola azul es la última en ser extraída.

Ahora, **Ana** y **Carlos** quieren escribir otra simulación, pero esta vez **con reemplazo**, para estimar la frecuencia de extracción de cada color en una gran cantidad de rondas de extracción. El grupo propone el siguiente código incompleto:

```
def simular_frecuencia_colores(n):
    prob_colores = __________   # (b)

    prob_distribution = make_array(prob_colores, __________)   # (c)

    simulated_proportions = __________   # (d)

    num_extracciones = __________ * simulated_proportions.__________   # (e) (f)

    return num_extracciones
```

2. (1.0 pt) Completa el espacio **(b)** con la probabilidad de extraer cualquier color de la bolsa.

3. (2.0 pt) Completa el espacio **(c)** con la probabilidad de extraer cualquier color que **no** sea azul.

4. (3.0 pt) Completa el espacio **(d)** con la función adecuada para generar proporciones de muestreo.

5. (2.0 pt) Completa el espacio **(e)** con el número total de extracciones.

6. (2.0 pt) Completa el espacio **(f)** con el índice correcto para acceder a la proporción de bolas azules.

3. (24 puntos) Registro de entrenamientos de Sofía

Cada día, **Sofía** realiza una sesión de entrenamiento y registra cuánto tiempo le toma completar cada rutina. Ella almacena estos datos en una tabla llamada **trainings**, donde cada fila representa una sesión de entrenamiento. A continuación, se muestra un ejemplo de la tabla para 10 dias de entrenmaiento, la tabla total tiene 120 dias de entrenamiento, que no es mostrada en su totalidad :

NumeroActividad	DiaEntrenamiento	Actividad	Horas	Minutos
1	Lunes	Pesas	1	22
2	Martes	Natación	2	31
3	Miércoles	Correr	2	38
4	Jueves	Bicicleta Estática	2	33
5	Viernes	Bicicleta Estática	1	21
6	Sábado	Natación	1	42
7	Lunes	Pesas	1	43
8	Martes	Correr	2	39
9	Miércoles	Bicicleta	1	15
10	Jueves	Natación	2	44

Table 1: Ejemplo de tabla de entrenamientos de Sofía

Donde,

- **NumeroActividad**: (`int`) Número de actividad que realizó
- **DiaEntrenamiento**: (`string`) el día de la semana en que se planeó el entrenamiento.
- **Actividad**: (`string`) Actividad que realiza a la semana
- **Minutos**: (`int`) la cantidad de minutos que tomó completar la sesión, redondeado hacia abajo.
- **Segundos**: (`int`) los segundos adicionales después de los minutos completos.

Sofía quiere analizar su entrenamiento y necesita trabajar con la tabla `training`, la cual es un objeto de tipo `datascience`

1. (2.0 pt) Dada la tabla `training` qué es una tabla de `datascience`, ¿cuál de las siguientes opciones permite seleccionar una columna particular correctamente?

   Seleccione **todas** las respuestas correctas.

   - ☐ `training.select("Minutos")`
   - ☐ `training.column("Minutos")`
   - ☐ `training("Minutos")`
   - ☐ `training_.("Minutos")`

2. (2.0 pt) Sofía quiere crear una nueva columna en `training` llamada `"TiempoTotal(min)"` que contenga el tiempo total de cada entrenamiento en **minutos**

   Para ello, define la siguiente función:

   ```
   def convertir_a_minutos(horas, minutos):
       return horas * 60 + minutos
   ```

   ¿Cuál de las siguientes opciones permite crear correctamente la nueva columna en la tabla usando el método `apply`? Seleccione **una o varias** respuestas correctas.

   - ☐ `training.with_column("TiempoTotal(min)", training.apply(convertir_a_minutos, "Horas", "Minutos"))`
   - ☐ `training.with_column("TiempoTotal(min)", convertir_a_minutos("Horas", "Minutos"))`
   - ☐ `training.apply("TiempoTotal(min)", convertir_a_minutos, "Horas", "Minutos")`
   - ☐ `training.apply("TiempoTotal(min)", convertir_a_minutos)`

3. (2.0 pt) Después de realizar la conversión, Sofía obtiene la siguiente tabla asignándole un nuevo nombre training_process:

NumeroActividad	DiaEntrenamiento	Actividad	Horas	Minutos	TiempoTotal(min)
1	Lunes	Pesas	1	22	82
2	Martes	Natación	2	31	151
3	Miércoles	Correr	2	38	158
4	Jueves	Bicicleta Estática	2	33	153
5	Viernes	Bicicleta Estática	1	21	81
6	Sábado	Natación	1	42	102
7	Lunes	Pesas	1	43	103
8	Martes	Correr	2	39	159
9	Miércoles	Bicicleta	1	15	75
10	Jueves	Natación	2	44	164

Table 2: Ejemplo de tabla de entrenamientos de Sofía, después de transformarla

¿Cuál sería la expresión en Python para calcular el **valor medio** del tiempo total de entrenamiento por actividad? Escriba su respuesta en la línea siguiente:

4. (2.0 pt) ¿Cuál sería la expresión en Python para calcular el **valor máximo** del tiempo total de entrenamiento por actividad? Escriba su respuesta en la línea siguiente:

5. (2.0 pt) ¿Cuál sería la expresión en Python para calcular el **valor mínimo** del tiempo total de entrenamiento por actividad?.Escriba su respuesta en la línea siguiente:

6. (2.0 pt) ¿Cuál de las siguientes expresiones en Python arroja la salida mostrada en la siguiente tabla?

   - ☐ `training_process.pivot("Actividad", "DiaEntrenamiento", "TiempoTotal(min)", np.mean)`
   - ☐ `training_process.group("Actividad", "DiaEntrenamiento", "TiempoTotal(min)", np.mean)`
   - ☐ `training_process.apply("Actividad", "DiaEntrenamiento", "TiempoTotal(min)", np.mean)`
   - ☐ `training_process.join("Actividad", "DiaEntrenamiento", "TiempoTotal(min)", np.mean)`

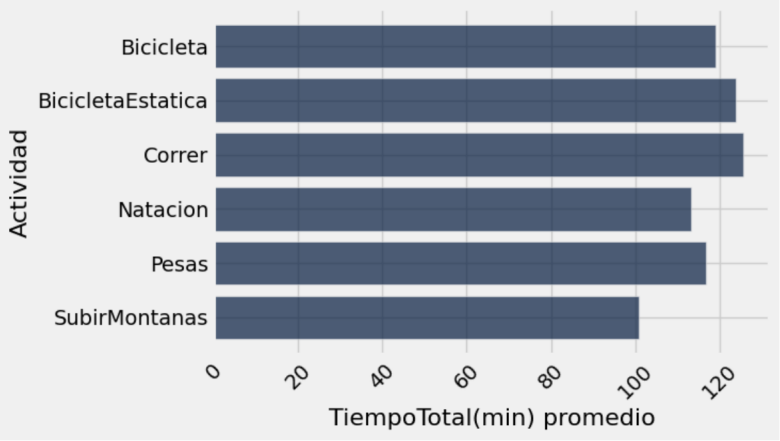
DiaEntrenamiento	Bicicleta	Bicicleta Estática	Correr	Natación	Pesas	Subir Montañas
Jueves	116.0	146.5	152.0	123.333	123.25	84.5
Lunes	137.5	98.0	95.0	124.5	112.667	104.0
Martes	126.333	155.5	111.75	116.0	102.667	66.5
Miércoles	113.167	96.5	136.333	102.5	154.0	106.5
Sábado	124.0	151.0	138.0	101.25	153.0	103.0
Viernes	106.0	107.6	145.0	104.5	90.5	119.667

Table 3: Tiempo total promedio (en minutos) de entrenamiento por actividad y día de la semana

7. (2.0 pt) Sofía quiere analizar el tiempo total promedio dedicado a cada actividad y ha generado el siguiente gráfico de barras: ¿Cuál de las siguientes opciones en Python genera correctamente el gráfico mostrado? Seleccione **todas** las respuestas correctas.

   - ☐ `training_process.select("Actividad", "TiempoTotal(min)").group("Actividad", np.mean).barh("Actividad")`

- ☐ `training_process.select("Actividad", "TiempoTotal(min)").group("Actividad", np.mean).bar("Actividad")`
- ☐ `training_process.group("Actividad", np.mean).bar("TiempoTotal(min)")`
- ☐ `training_process.bar("Actividad", "TiempoTotal(min)")`



8. (8.0 pt) Sofía quiere analizar si el tiempo total promedio dedicado a la actividad **Subir Montañas** es consistente con la variabilidad aleatoria o si presenta un patrón significativo. Para ello, debe formular una hipótesis y realizar un análisis estadístico.

   Con base en el gráfico, plantee una hipótesis nula ($\mathbf{H}_0$) y una hipótesis alternativa ($\mathbf{H}_A$) para evaluar si el tiempo total de entrenamiento en **Subir Montañas** es resultado de la aleatoriedad.

   Hipótesis Nula ($\mathbf{H}_0$):

   Hipótesis Alternativa ($\mathbf{H}_A$):

9. (2.0 pt) Después de realizar la prueba de permutación, Sofía obtiene un valor p de **0.5**. ¿Qué conclusión se puede sacar a partir de este resultado?

4. (29 puntos) Peso bebes al nacer

El siguiente histograma muestra la distribución del peso al nacer de bebés cuyas madres eran fumadoras (color amarillo) y no fumadoras (color azul). Basado en la información visual proporcionada, ¿cuál de las siguientes afirmaciones es correcta?

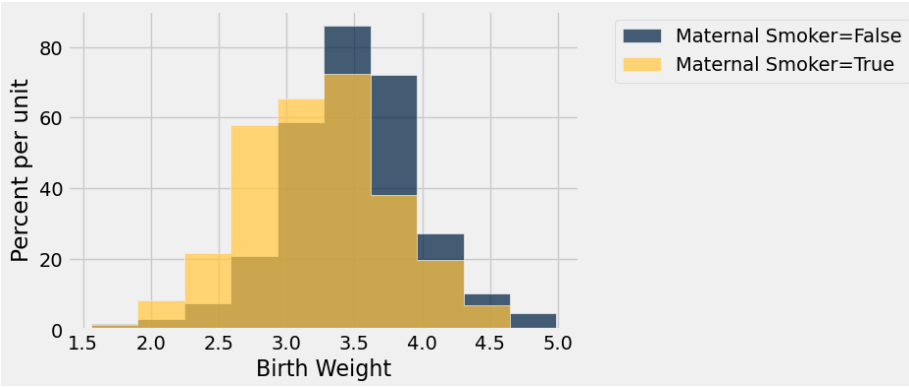


Figure 1: Distribución del peso al nacer según si la madre es fumadora o no.

1. (5.0 pt) Considerando la naturaleza de los histogramas y el gráfico mostrado, ¿cuáles de las siguientes afirmaciones son correctas? Selecciona una o varias opciones.

   - ☐ El histograma muestra la distribución de una variable numérica (peso al nacer) separada en intervalos, con la altura de las barras representando la densidad o frecuencia relativa de los datos en cada intervalo.
   - ☐ La forma del histograma sugiere que los bebés de madres fumadoras tienen una mayor probabilidad de nacer con menor peso en comparación con los de madres no fumadoras.
   - ☐ Como ambos histogramas tienen una distribución con forma similar, podemos concluir que la media y la desviación estándar del peso son idénticas para ambos grupos.
   - ☐ Un histograma no muestra datos individuales, sino una representación agregada de la distribución de los valores dentro de cada intervalo.
   - ☐ El área total de cada histograma representa el porcentaje total de observaciones en cada grupo, lo que permite comparar la proporción de pesos entre bebés de madres fumadoras y no fumadoras.

2. (4.0 pt) ¿Qué observaciones se pueden hacer a partir del histograma de pesos al nacer? Selecciona una o varias opciones.

   - ☐ Los bebés de madres fumadoras tienden a tener un peso menor al nacer que los de madres no fumadoras.
   - ☐ Hay más bebés de madres no fumadoras con pesos mayores a 3.5 kg en comparación con los de madres fumadoras.
   - ☐ No hay diferencias significativas en la distribución de peso entre ambos grupos.
   - ☐ Los pesos de los bebés de madres fumadoras y no fumadoras siguen exactamente la misma distribución.

3. (5.0 pt) ¿Cuáles de las siguientes afirmaciones son correctas sobre la distribución de pesos al nacer?. Selecciona una o varias opciones.

   - ☐ La mayoría de los bebés de madres fumadoras tienen un peso entre 2.5 y 3.5 kg.
   - ☐ La curva de los bebés de madres no fumadoras está desplazada hacia la derecha en comparación con la de los bebés de madres fumadoras.
   - ☐ No hay bebés con peso superior a 4.5 kg en el conjunto de datos.

- ☐ El porcentaje de bebés con peso inferior a 2.5 kg es mayor en madres fumadoras que en madres no fumadoras.

A continuación, se presentan una hipótesis nula y una hipótesis alternativa sobre la relación entre el peso al nacer y el hábito de fumar de la madre. Responde la pregunta seleccionando todas las opciones correctas.

Hipótesis: Hipótesis nula (H_0): En la población, la distribución de los pesos natales de los bebés es la misma para las madres que no fuman que para las madres que lo hacen. La diferencia en la muestra se debe al azar.

Hipótesis alternativa (H_A): En la población, los bebés de las madres que fuman tienen un peso al nacer más bajo, en promedio, que los bebés de las madres que no fuman.

4. (5.0 pt) Si realizamos una prueba de hipótesis basada en estos datos, ¿cuáles de las siguientes afirmaciones son correctas?

   - ☐ La hipótesis nula supone que cualquier diferencia en el peso al nacer observada en la muestra es consecuencia de la variabilidad natural y no de un efecto real del hábito de fumar.
   - ☐ Si el p-valor obtenido en la prueba es muy pequeño (por ejemplo, menor a 0.05), rechazamos la hipótesis nula en favor de la hipótesis alternativa.
   - ☐ Si la hipótesis alternativa es cierta, esperaríamos que la media del peso al nacer de los bebés de madres fumadoras sea menor que la de los bebés de madres no fumadoras en la población.
   - ☐ Un p-valor grande sugiere que la diferencia observada en la muestra podría explicarse por el azar, por lo que no hay suficiente evidencia para rechazar la hipótesis nula.
   - ☐ La prueba de hipótesis nos permitirá probar con certeza si fumar causa una reducción en el peso al nacer.

5. (4.0 pt) Completa los espacios en blanco en el siguiente código de Python para realizar una simulación de diferencias de medias en 1000 permutaciones.

```
def diferencia_medias(table, group_label):
    reduced = table.select("Birth Weight", group_label)
    means_table =  reduced.group(group_label, np.average)
    means = means_table.column(1)
    return means.item(1) - means.item(0)

def diferentes_muestras():
    shuffled_labels = fumador_peso_nacido.sample(with_replacement=False).column(0)
    original_and_shuffled = fumador_peso_nacido.with_column("Shuffled Label", shuffled_labels)
    shuffled_only = original_and_shuffled.select('Birth Weight', 'Shuffled Label')
    shuffled_group_means = ________________ # (a) Completa esta línea
    return diferencia_medias(______________, "Shuffled Label") # (b) Completa esta línea

dif = make_array()
repetition = 1000
for i in range(0, repetition):
    dif = np.append(dif, diferentes_muestras())
```

i. ¿Qué código debería ir en (a) para calcular la media del peso al nacer en cada grupo de fumadoras y no fumadoras? ii. ¿Qué argumento falta en (b) para llamar correctamente a la función `diferencia_medias`?

Espacios para respuestas:

(a)

(b)

6. (7.0 pt) Observa el siguiente histograma y responde la pregunta seleccionando todas las opciones correctas.

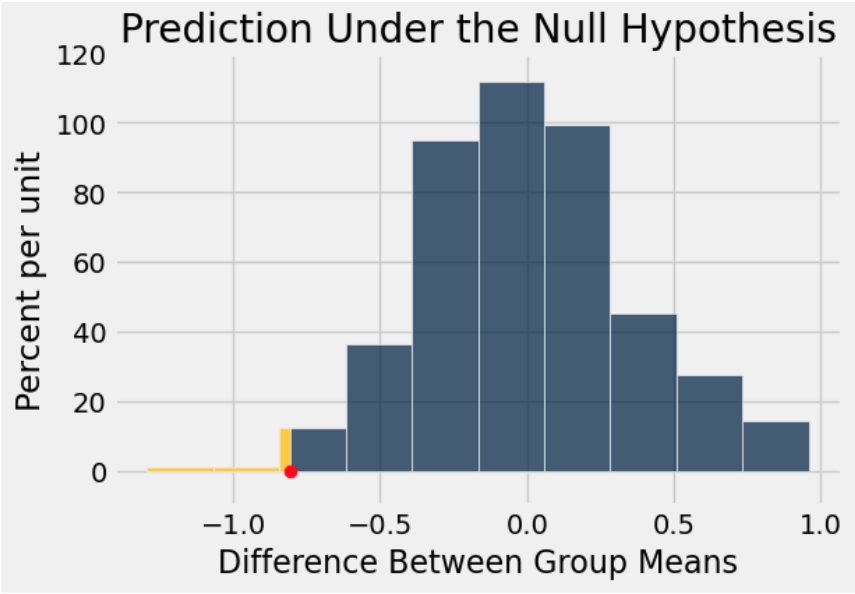


Figure 2: Distribución de diferencias de medias bajo la hipótesis nula.

Código de cálculo del p-valor empírico

```
empirical_p = np.count_nonzero(age_differences <= observed_age_difference) / nsimulations
p=empirical_p * 100
print(p)
```

Supongamos que al ejecutar el código obtenemos $p = 0.06$.

Considerando la distribución simulada bajo la hipótesis nula y el cálculo del p-valor, ¿cuáles de las siguientes afirmaciones son correctas?

- ☐ Un p-valor de 0.06 significa que en el 6% de las simulaciones la diferencia de medias fue tan extrema o más extrema que la observada en los datos reales.
- ☐ Un p-valor de 0.06 proporciona evidencia fuerte en contra de la hipótesis nula.
- ☐ Si el p-valor fuera menor que 0.05, podríamos concluir que hay suficiente evidencia para rechazar la hipótesis nula en favor de la alternativa.
- ☐ Un p-valor de 0.06 indica que la diferencia observada podría deberse al azar, pero está cerca del umbral de significancia y podría considerarse en contexto con otros factores.

Principales Comandos de la Librería `datascience`

Comando	**Descripción**
`Table.read_table("archivo.csv")`	Carga un archivo CSV como una tabla de datos.
`tabla.show(n)`	Muestra las primeras n filas de la tabla.
`tabla.select("columna1", "columna2")`	Crea una nueva tabla con solo las columnas seleccionadas.
`tabla.drop("columna")`	Crea una nueva tabla eliminando la columna indicada.
`tabla.sort("columna", descending=True)`	Ordena la tabla por la columna especificada en orden descendente.
`tabla.where("columna", are.equal_to(valor))`	Filtra la tabla y muestra solo las filas donde la columna cumple la condición dada.
`tabla.group("columna", np.sum)`	Agrupa los valores por una columna y aplica una función de agregación (ejemplo: suma).
`tabla.apply(funcion, "columna")`	Aplica una función a cada fila de la columna seleccionada.
`tabla.hist("columna")`	Dibuja un histograma de la columna indicada.
`tabla.scatter("columna1", "columna2")`	Genera un gráfico de dispersión entre dos columnas.
`tabla.num_rows`	Devuelve el número de filas en la tabla.
`tabla.num_columns`	Devuelve el número de columnas en la tabla.
`tabla.column("columna")`	Devuelve los valores de una columna como un array.
`tabla.item(fila, "columna")`	Devuelve el valor específico de una celda en la tabla.
`tabla.relabeled("columna_vieja", "columna_nueva")`	Renombra una columna de la tabla.
`tabla.simple_proportion("columna", valor)`	Calcula la proporción de valores iguales a un valor dado.
`tabla.sample(k)`	Selecciona k filas aleatorias de la tabla (con reemplazo por defecto).

Guía Básica de Python

Concepto	**Ejemplo en Python**
Definir una variable	`x = 10`
Listas en Python	`numeros = [1, 2, 3, 4, 5]`
Acceder a un elemento de una lista	`numeros[0]` devuelve 1
Agregar un elemento a una lista	`numeros.append(6)`
Eliminar un elemento de una lista	`numeros.remove(3)`
Estructura condicional	`if x > 5:  print("Mayor a 5")`
Bucle `for`	`for i in range(5):  print(i)`
Bucle `while`	`while x > 0:  x -= 1`
Definir una función	`def suma(a, b):  return a + b`
Importar librerías	`import numpy as np`
Crear un array de NumPy	`arr = np.array([1, 2, 3])`
Filtrar una lista con `list comprehension`	`pares = [x for x in numeros if x % 2 == 0]`
Crear un diccionario	`dicc = {"a":  1, "b":  2}`
Acceder a un valor en un diccionario	`dicc["a"]` devuelve 1
Leer un archivo CSV con Pandas	`import pandas as pd` `df = pd.read_csv("datos.csv")`
Obtener estadísticas descriptivas	`df.describe()`
Filtrar un DataFrame de Pandas	`df[df["columna"] > 10]`